SVKM'S NMIMS
SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING
INDORE



MINI PROJECT REPORT

ON

IoT Smart Irrigation Sensor & Warning System

Submitted in partial fulfillment of the requirements for the course

IoT and Applications

SUBMITTED BY

Shubhpreet Singh Verma | SAP ID: 70562400138
Anvi Jain | SAP ID: 70562400141
Eshita Agicha | SAP ID: 70562400160

B.Tech (AI & DS), Semester V, Division: B

SUBMITTED TO

Dr. Nirmal Kumar Gupta
Associate Professor

Academic Year: 2026–2027

CERTIFICATE

This is to certify that the mini project report entitled **"IoT Environmental Monitor & Warning System"** is a bonafide record of the work carried out by **Shubhpreet Singh Verma**, **Anvi Jain, Eshita Agicha** (SAP ID: 70562400138, 70562400141, 70562400160), B.Tech, Semester V, Division B, at the School of Technology Management & Engineering, SVKM's NMIMS, Indore, in partial fulfillment of the requirements of the course **IoT and Applications** during the academic year 2026–2027.

The work reported here was carried out under the guidance of the undersigned and, to the best of my knowledge, has not been submitted elsewhere for any other course, degree or credit.

Date: 6/10/2026
Place: Indore

_____

Dr. Nirmal Kumar Gupta
Associate Professor
IoT and Applications

Declaration by the Student

We hereby declare that this report is the result of our own work, written in our own words. All books, papers, datasheets and online documentation consulted have been acknowledged in the References section. A plagiarism report for this document is attached at the end of the report.

_____

Shubhpreet Singh Verma
Anvi Jain
Eshita Agicha

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **Dr. Nirmal Kumar Gupta**, Associate Professor, for his guidance and encouragement during this mini project for the course IoT and Applications. His feedback and the practical orientation of the course helped us turn an idea into a working prototype.

We are thankful to **SVKM's NMIMS, School of Technology Management & Engineering, Indore**, for providing the academic environment and facilities that made this work possible.

We also thank our classmates and friends for the discussions that helped us sharpen the design, and our families for their constant support and patience.

Finally, we acknowledge the open-source and developer communities behind the ESP32 Arduino core, PlatformIO, the DHT sensor library and the Blynk platform. Their documentation and tools lowered the barrier to building and testing this system.

ABSTRACT

Monitoring temperature, humidity, rainfall, soil wetness and water level matters in agriculture, flood preparedness and building management, yet manual observation is slow and cannot raise an instant warning. This mini project presents an IoT Environmental Monitor and Warning System built around the ESP32 DevKit V1. A DHT22 sensor supplies temperature and relative humidity, a water-level sensor and an FC-28 soil probe give analog readings on ADC1 pins, and a rain module gives a digital wet or dry signal. The firmware, written in C++ with the Arduino framework and developed in VS Code with PlatformIO, compares each reading with fixed thresholds and applies a highest-severity rule to grade the overall situation as Normal, Warning or Danger. The grade is shown immediately on green, yellow and red LEDs, and a passive buzzer driven by a PWM tone gives a short beep in the Danger state. At the same time, the ESP32 publishes every reading to the Blynk cloud over Wi-Fi every two seconds through virtual pins V0 to V4 for remote viewing. The prototype shows that a low-cost node can combine multi-sensor acquisition, decision making at the edge and cloud visualisation. The thresholds are practical prototype values taken from observed readings rather than laboratory-calibrated figures, which is recognised as the main limitation and a clear direction for future work.

Keywords: Internet of Things, ESP32, Environmental Monitoring, Early Warning System, Blynk, Sensor Fusion

TABLE OF CONTENTS

Chapter 1: Introduction....1
1.1 Background.... 1
1.2 Problem Statement.... 1
1.3 Objectives....1
1.4 Scope....2
1.5 Motivation....2
Chapter 2: Literature Review....3
2.1 Overview....3
2.2 Reviewed Works....3
2.3 Comparative Summary....4
2.4 Research Gap....4
Chapter 3: System Design and Architecture....5
3.1 Proposed System....5
3.2 System Architecture....5
3.3 Hardware Requirements....6
3.4 Software Requirements....7
3.5 Pin Allocation and Circuit Diagram....7
3.6 Working Principle and Warning Logic....8
Chapter 4: Implementation....10
4.1 Methodology....10
4.2 Hardware Setup....10
4.3 Software Implementation....11
4.4 Communication Protocol....14
4.5 IoT Platform: Blynk....14
Chapter 5: Results and Discussion....15
5.1 Experimental Setup....15
5.2 Output Screenshots....15
5.3 Test Cases....16
5.4 Observations....17
5.5 Discussion....18
5.6 Limitations Observed During Testing....19
Chapter 6: Conclusion and Future Scope....20
6.1 Summary of Achievements....20
6.2 Conclusion....20
6.3 Limitations....20
6.4 Future Enhancements....21
References....22
Appendix....23
Appendix A: Complete Source Code....23
Appendix B: Quick Reference....23
Plagiarism Report....24

LIST OF FIGURES AND TABLES

Table 1: Comparison of the reviewed systems with the proposed system..... 4
Figure 1: Layered block diagram of the IoT Environmental Monitor..... 5
Table 2: Hardware components used.....6
Table 3: Software tools and libraries.....7
Table 4: Final GPIO pin mapping..... 8
Figure 2: Circuit connection diagram showing the GPIO pin mapping.....8
Table 5: Warning thresholds.....9
Table 6: Local alert behaviour.....9
Table 7: PlatformIO project files.....11
Figure 3: Program flow of the firmware..... 12
Table 8: Important statements used in the firmware..... 12
Table 9: Blynk virtual pin assignment.....14
Figure 4: Assembled hardware prototype..... 15
Figure 5: Serial monitor output showing live sensor readings.....16
Figure 6: Blynk dashboard with temperature, humidity, water, soil and rain widgets.....16
Table 10: Test cases and expected system response.....17
Table 11: Observed sensor behaviour.....17
Table 12: Problems encountered and their resolution..... 18
Table A1: Summary of configuration values..... 23

Chapter 1: Introduction

1.1 Background

Air temperature, humidity, rainfall, soil wetness and water level are the everyday variables that decide whether a crop thrives, whether a storeroom stays dry and whether a low-lying area stays safe. Traditionally these quantities were checked by hand. Manual checks are irregular, depend on someone being present, and give no warning in the moment when a condition turns dangerous.

The Internet of Things (IoT) addresses this gap by giving ordinary objects the ability to sense their surroundings, process the data and exchange it over a network without constant human attention. Inexpensive Wi-Fi-enabled microcontrollers such as the ESP32 [6] and ready-made cloud dashboards such as Blynk [7] now allow a student-built node to deliver monitoring that once needed a commercial data logger.

This mini project builds one such node. It reads five environmental parameters, interprets them on the device, shows the result on local indicators, and mirrors the same data to a cloud dashboard.

1.2 Problem Statement

Many small monitoring prototypes either display raw numbers only or concentrate on a single hazard such as flooding or irrigation. A person looking at a column of numbers still has to decide whether the situation is serious, and a dashboard is of no help to someone standing next to the equipment when the network is slow or the phone is out of reach.

There is therefore a need for a compact, low-cost device that observes several parameters together, converts them into one easily understood severity level, signals that level at the site through lights and sound, and also reports it remotely.

1.3 Objectives

1. To measure temperature, humidity, water level, soil moisture and rain with a single ESP32-based node.
2. To acquire digital, single-wire and analog sensor signals on the ESP32 and compare them with defined thresholds.

3. To grade the overall condition as **Normal**, **Warning** or **Danger** using a highest-severity rule.
4. To give immediate local indication with green, yellow and red LEDs and an audible buzzer alert in the Danger state.
5. To publish the readings every two seconds to the Blynk cloud through virtual pins V0 to V4 for remote monitoring.
6. To document the practical problems met during development and the calibration limits of the prototype.

1.4 Scope

The project covers the design, assembly, programming and testing of a breadboard prototype that monitors and warns. The thresholds are fixed in firmware and were chosen from observed sensor behaviour.

The system is **not** an automatic irrigation or flood-control device: it has no pump, valve or relay and takes no physical action on the environment. Long-term data logging, a weatherproof enclosure, battery operation and laboratory-grade calibration are outside the present scope and are listed as future work in Chapter 6.

1.5 Motivation

Three reasons motivated this work. First, it brings together embedded programming, analog and digital interfacing, wireless networking and cloud services in one practical build, which is the core of the IoT and Applications course. Second, the parameters chosen map directly onto real needs: soil moisture and temperature for farming, rain and water level for flood awareness, and humidity for indoor comfort and storage. Third, a warning that is visible and audible at the spot is more useful than a dashboard alone, so the design deliberately keeps the decision logic on the device and treats the cloud as a second channel.

Chapter 2: Literature Review

2.1 Overview

Five published works that use the ESP32 family with the Blynk platform were reviewed. They fall into three application areas whose parameters this project combines: general environmental monitoring [1], [2], farm and plant irrigation [3], [4], and flood warning [5]. The limitations noted below are relative to the aims of this project, not criticisms of the original authors.

2.2 Reviewed Works

Babic et al. [1] propose a simple real-time environmental monitoring node in which an ESP32 collects air temperature, humidity, air pressure and dust-like particle concentration, and Blynk Cloud stores, manages and visualises the data. The node was installed at a university building and run alongside a commercial ecological monitoring system. The work concentrates on atmospheric quantities; water, soil and rain sensing and graded local alerts are not described.

Adeagbo [2] builds an ESP32 and DHT22 weather node whose temperature and humidity appear on Blynk virtual pins V0 and V1. The device was tested indoors and outdoors and compared with official weather figures using percentage error; the humidity difference was small, whereas the temperature difference was clearly larger. The system uses only two parameters and offers no hardware alert, and the author lists rainfall and other sensors as future additions.

Faysal and Mohammed [3] describe a remote farm monitoring and irrigation system in which an ESP32 reads a DHT22 and a soil moisture sensor, shows the data on a Blynk smartphone dashboard, and switches a water pump automatically. The study is aimed at crops and horticulture and notes that each plant needs its own sensor and pump. Its focus is irrigation control, and rain or water-level conditions are outside the described scope.

Mohd Najid and Omar [4] present an IoT plant-watering prototype on an ESP32 board with soil moisture, DHT11 and pH sensors and a Blynk application. Water and liquid fertiliser are supplied automatically, and an experiment on eggplants compared smart and conventional irrigation over five weeks, with the smart system performing better. It is a small-scale plant study, and the DHT11 is a coarser sensor than the DHT22 [2].

Lias and Kumaran [5] develop an ESP32 flood monitor that measures water depth with a TOF10120 laser distance sensor, shows it on an LCD, adds an ESP32-CAM for visual checking, and sends Blynk notifications. Three LEDs and a buzzer indicate three severity bands of water depth. This is the closest relative of the present work in its use of coloured LEDs, a buzzer and severity bands, but it observes a single hazard and has no soil, temperature or humidity context.

2.3 Comparative Summary

Table 1: Comparison of the reviewed systems with the proposed system

Ref.	Platform	Parameters sensed	Output / action	Limitation relative to this work
[1]	ESP32 + Blynk Cloud	Temperature, humidity, air pressure, dust-like particles	Cloud dashboard; compared with a commercial monitor	No water, soil or rain sensing; no graded local alert
[2]	ESP32 + DHT22 + Blynk	Temperature, humidity	Blynk dashboard on virtual pins V0 and V1	Only two parameters; larger temperature error reported
[3]	ESP32 + Blynk smartphone app	Temperature, humidity, soil moisture	Dashboard plus automatic pump irrigation	Irrigation-oriented; no rain or water-level hazard handling
[4]	NodeMCU ESP32 + Blynk	Soil moisture, humidity (DHT11), pH	Automatic watering and liquid fertiliser	Small-scale plant study; not a warning design
[5]	ESP32 + Blynk notifications	Water depth (laser distance sensor)	Three LEDs, buzzer, LCD, camera, notifications	Single hazard; no soil, temperature or humidity context
This work	ESP32 + Blynk	Temperature, humidity, water level, soil moisture, rain	Normal / Warning / Danger grading; LEDs, buzzer and dashboard	Fixed prototype thresholds; no actuation

2.4 Research Gap

The reviewed systems each emphasise one application. The environmental monitors [1], [2] sense atmospheric variables, the irrigation systems [3], [4] concentrate on soil and plants, and the flood system [5] concentrates on water depth. Only [5] gives a graded local alert, and only for one quantity.

The gap addressed here is integration: one inexpensive node that senses temperature, humidity, water level, soil moisture and rain, fuses them into one Normal, Warning or Danger level through a transparent highest-severity rule, announces that level locally with LEDs and

a buzzer, and streams the same readings to a cloud dashboard. The contribution is a documented, reproducible combination of well-understood parts, not a new sensing technique.

Chapter 3: System Design and Architecture

3.1 Proposed System

The proposed system is an ESP32-based monitoring and warning node. Four sensing modules feed the microcontroller: a DHT22 for temperature and humidity, a water-level sensor, an FC-28 soil moisture probe and a rain-detection module. The ESP32 samples them periodically, compares the values with predefined limits, and sets an overall severity level. Three LEDs and a buzzer present that level locally, while the on-board Wi-Fi radio carries the raw readings to a Blynk dashboard.

3.2 System Architecture

Figure 1 shows the architecture as a set of layers. The **sensing layer** converts physical conditions into electrical signals. The **processing (edge) layer** is the ESP32, which reads the signals, applies the warning logic and drives the outputs. The **local alert layer** consists of the LEDs and buzzer. The **network and application layers** are the Wi-Fi link, the Blynk Cloud and the dashboard. Because the decision is taken on the ESP32 itself, the node is an edge device: it does not need the cloud to decide what the situation is.

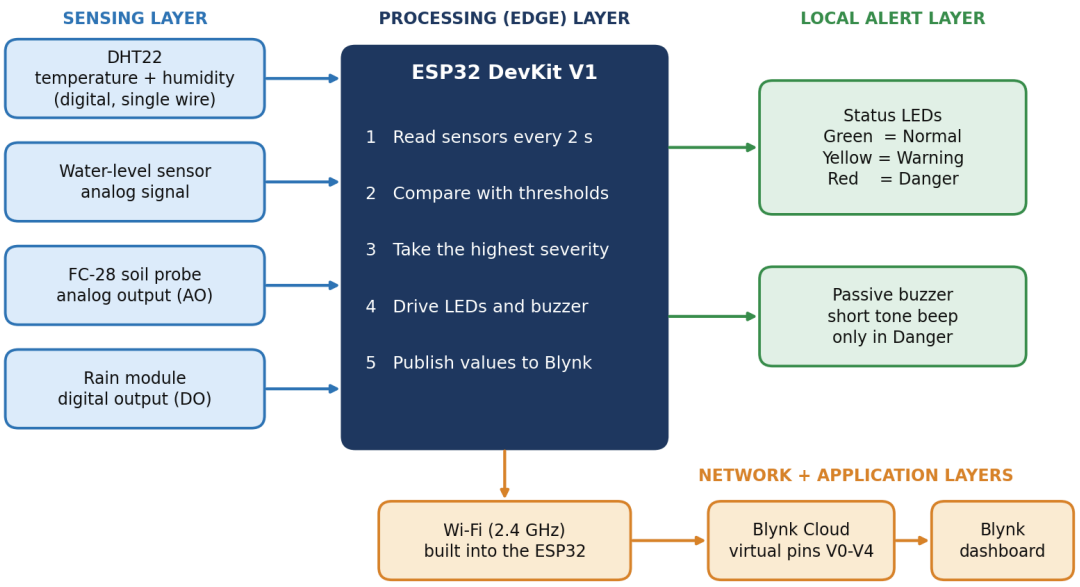


Figure 1: Layered block diagram of the IoT Environmental Monitor

3.3 Hardware Requirements

Table 2: Hardware components used

S. No.	Component	Role / key point	Connection
1	ESP32 DevKit V1	Main controller; built-in Wi-Fi; 3.3 V logic; 12-bit ADC	USB; GPIO pins
2	DHT22 sensor	Digital temperature and relative-humidity sensor	Data to GPIO15
3	Water-level sensor	Analog signal that changes with water contact	S pin to GPIO34
4	FC-28 soil moisture probe with module	Resistive probe; analog output indicates wetness	AO to GPIO35
5	Rain sensor module	Sensing plate with comparator board; digital wet/dry output	DO to GPIO27
6	Red, yellow and green LEDs	Danger, Warning and Normal indicators	GPIO25, 26, 33
7	220 Ω resistors (3)	Series current limiting for the LEDs	In series with LEDs
8	Passive buzzer	Audible alert, driven with a tone (PWM)	GPIO32
9	Breadboard and jumper wires	Solderless prototyping with a common ground rail	—
10	USB cable and laptop	Power, firmware upload and serial monitoring	USB

ESP32 DevKit V1. The ESP32 integrates Wi-Fi and Bluetooth on one chip [6], has a dual-core 32-bit processor, and offers a 12-bit analog-to-digital converter that returns counts from 0 to 4095. Its GPIO pins work at 3.3 V, so every signal connected to an input must stay within that level.

DHT22. The DHT22 combines a capacitive humidity element and a thermistor and sends its result as a digital signal on one data line, so a library can handle the protocol [2], [8]. The device can supply fresh data roughly every two seconds [2], which matches the two-second update period used in this project.

Water-level and rain sensors. Both boards carry exposed parallel copper traces. Water bridging the traces changes the electrical resistance between them, so the output varies with how much of the board is wet. The water-level board is read as an analog value. The rain board has a comparator with an adjustable potentiometer, and the project uses its digital output, which reads 0 when rain is detected and 1 otherwise.

FC-28 soil probe. The probe is a two-prong resistive sensor: wetter soil conducts better between the prongs. Its comparator module offers an analog output (AO) and a digital output (DO), and only AO is used. In the prototype, values close to 4095 appeared in air or very dry conditions and roughly 1250 to 1350 when the probe was wet.

LEDs, resistors and buzzer. Each LED is protected by a 220 Ω resistor. With a 3.3 V output and a forward voltage of about 2 V, the current is $I = (3.3 - 2.0) / 220 \approx 5.9$ mA, which is comfortably within what a GPIO pin can supply. The buzzer behaved as a passive type, which contains no oscillator of its own, so it must be driven by a frequency (tone) instead of a steady HIGH level.

3.4 Software Requirements

Table 3: Software tools and libraries

Item	Purpose
Visual Studio Code with PlatformIO	Editor, build system, uploader and serial monitor for the project
Arduino framework for ESP32	Provides pinMode, digitalRead, analogRead, PWM and Wi-Fi APIs in C++
`WiFi.h`	Wi-Fi connectivity of the ESP32
`BlynkSimpleEsp32.h`	Connection to Blynk Cloud, virtual pins and BlynkTimer
`DHT.h`	Driver for the DHT22 sensor
Blynk IoT cloud and dashboard	Remote display of readings on datastreams V0 to V4
Serial monitor at 115200 baud	Debug output and verification on the development computer

3.5 Pin Allocation and Circuit Diagram

Table 4 lists the final pin mapping, and Figure 2 presents the connections as a diagram. All grounds (ESP32, sensor modules, LED cathodes through their resistors, and buzzer) are tied to one common ground rail, which is essential for the analog readings to be meaningful.

Table 4: Final GPIO pin mapping

Device	ESP32 GPIO	Signal type
DHT22 data	15	Digital (single wire)
Water sensor S	34	Analog input (ADC1)
Soil probe AO	35	Analog input (ADC1)
Rain module DO	27	Digital input
Red LED	25	Digital output

Device	ESP32 GPIO	Signal type
Yellow LED	26	Digital output
Green LED	33	Digital output
Buzzer	32	PWM / tone output

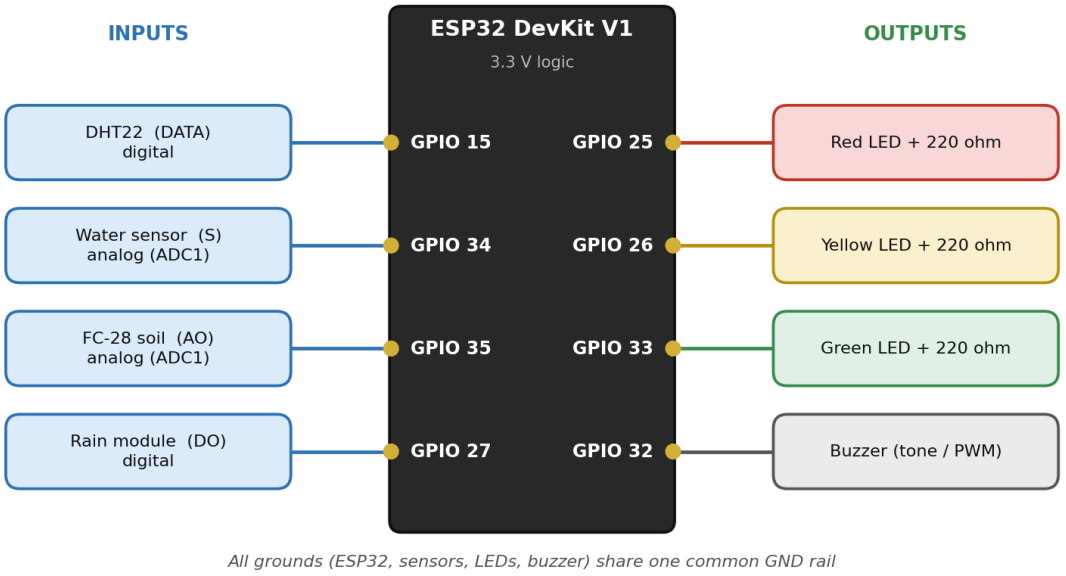


Figure 2: Circuit connection diagram showing the GPIO pin mapping

The allocation follows a few rules. GPIO34 and GPIO35 are input-only pins that belong to ADC1, which suits analog sensors and, unlike ADC2, remains usable while Wi-Fi is active [10]. The rain output is a plain digital level, so GPIO27 is sufficient. The outputs use ordinary general-purpose pins. One design caution concerns GPIO15, which carries the DHT22: it is a strapping pin that the ESP32 samples during reset, so if boot problems ever appear, relocating the sensor to a non-strapping pin is the first remedy to try.

3.6 Working Principle and Warning Logic

In operation the ESP32 reads every sensor once per cycle, grades each parameter as Normal, Warning or Danger using the limits in Table 5, and then takes the **highest** grade as the overall level. A single Danger reading therefore makes the whole system Danger, however calm the other parameters are. Humidity is displayed on the dashboard but does not influence the level.

Table 5: Warning thresholds

Parameter	Normal	Warning	Danger
Temperature	< 30 °C	30 to 34.99 °C	≥ 35 °C

Parameter	Normal	Warning	Danger
Water level (ADC count)	≤ 700	701 to 1500	> 1500
Soil (ADC count)	≤ 1999	2000 to 3500	> 3500
Rain (digital)	1 (no rain)	0 (rain detected)	Not treated as danger

Because the FC-28 gives high counts when dry, a soil value above 3500 represents very dry soil, which the system treats as Danger. Table 6 shows the local response for each level.

Table 6: Local alert behaviour

Level	Green LED	Yellow LED	Red LED	Buzzer
0 – Normal	ON	OFF	OFF	OFF
1 – Warning	OFF	ON	OFF	OFF
2 – Danger	OFF	OFF	ON	Short beep

Chapter 4: Implementation

4.1 Methodology

The project followed an incremental build-and-test approach so that each block could be verified before the next was added:

1. **Requirement analysis:** decide the parameters to monitor and the form of warning (Normal, Warning, Danger).
2. **Component selection:** choose the ESP32 for its built-in Wi-Fi and ADC, and low-cost sensors for each parameter.
3. **Breadboard assembly:** wire each sensor and output to the pins in Table 4 with a common ground.
4. **Firmware development:** write and test sensor reading, threshold logic and output control with serial printouts.
5. **Cloud configuration:** create the Blynk template and datastreams and connect the firmware to them.
6. **Testing and refinement:** exercise each sensor, correct problems, and fix the demonstration thresholds.

4.2 Hardware Setup

The sensors and indicators were assembled on solderless breadboards and connected to the ESP32 with jumper wires (the assembled prototype appears in Figure 4). The following practices were applied:

- Every module shares one ground rail with the ESP32; without this, analog readings drift and digital levels become unreliable.
- Each LED has its own 220 Ω series resistor, so no GPIO pin drives an LED directly.
- Analog outputs go only to ADC1 pins (GPIO34 and GPIO35). The sensor modules should be powered so that their analog output never exceeds the 3.3 V limit of the ESP32 inputs.
- The buzzer is connected to a PWM-capable output so that it can be driven with a tone.
- During development the board is powered and programmed through its USB connection.

4.3 Software Implementation

4.3.1 Development environment and project structure

The firmware is a PlatformIO project in VS Code using the Arduino framework for the ESP32. PlatformIO [9] downloads the board platform and libraries, compiles the code, uploads it and opens the serial monitor, so the whole workflow stays in one tool. Table 7 summarises the project files.

Table 7: PlatformIO project files

File / folder	Purpose
`platformio.ini`	Project configuration: board, framework, libraries, monitor speed
`src/main.cpp`	Main program containing setup(), loop() and the helper functions
`.pio/`	Generated build files; may be deleted and will be recreated
`lib/, include/, test/`	Optional project libraries, headers and tests (not required here)

4.3.2 Configuration and libraries

The top of the program defines the Blynk template details, includes the required libraries and declares the DHT22 pin and type. The credential values below are redacted; the real authentication token must never be shared or committed to a public repository.

Listing 1: Configuration section of main.cpp (credentials redacted)

```
#define BLYNK_TEMPLATE_ID   "<template-id>"
#define BLYNK_TEMPLATE_NAME "IoT Environmental Monitor"
#define BLYNK_AUTH_TOKEN    "<redacted>"

#include <Arduino.h>
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>

#define DHT_PIN  15
#define DHT_TYPE DHT22
```

4.3.3 Program flow

Figure 3 shows the flow of the program. In `setup()` the serial port, the DHT22, the pins, the buzzer PWM channel and the Blynk connection are initialised. The `loop()` function does very little: it keeps the Blynk connection alive and lets a BlynkTimer run. Every two seconds the timer calls `sendSensorData()`, which reads all sensors, publishes the values, computes the warning level and updates the outputs. Using a timer instead of a long `delay()` keeps

the loop responsive, and it also respects the Blynk guidance that values should not be written from inside the main loop, where they can flood the connection [11].

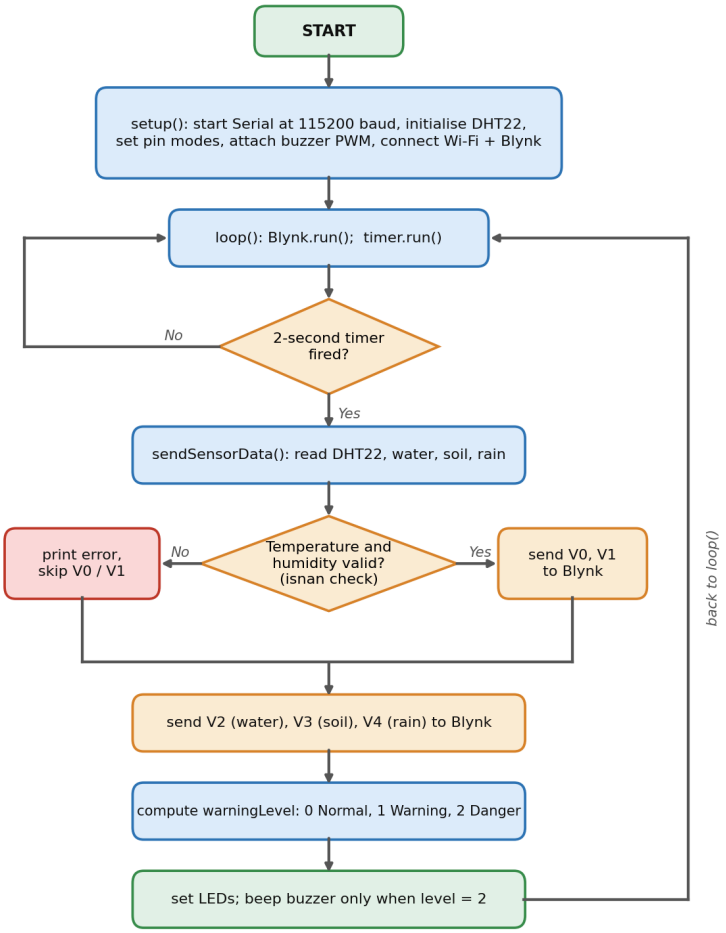


Figure 3: Program flow of the firmware

4.3.4 Key functions

Table 8: Important statements used in the firmware

Statement	Role in the program
`Serial.begin(115200)`	Starts the serial link used for debug output
`dht.begin()`	Prepares the DHT22 driver
`pinMode(pin, mode)`	Declares LED, buzzer and rain pins as outputs or inputs
`analogRead(pin)`	Returns a 0–4095 ADC count for the water and soil sensors
`digitalRead(pin)`	Returns 0 or 1 for the rain module output
`digitalWrite(pin, level)`	Switches the LEDs on or off
`Blynk.begin(auth, ssid, pass)`	Joins Wi-Fi and then connects to Blynk Cloud
`Blynk.virtualWrite(Vx, value)`	Sends a value to datastream Vx

Statement	Role in the program
`Blynk.run()`	Services the cloud connection on every loop pass
`timer.setInterval(2000L, fn)`	Schedules the sensor routine every 2 seconds
`timer.run()`	Executes any callback that has become due
`isnan(value)`	Detects an invalid floating-point DHT22 reading
`ledcWriteTone(...)`	Produces the buzzer tone through PWM

4.3.5 Classification and output logic

Listing 2 shows the logic that grades the readings, written to reproduce the thresholds of Table 5. Each parameter can only raise the level, never lower it, which implements the highest-severity rule.

Listing 2: Warning classification (representative implementation)

```
// returns 0 = Normal, 1 = Warning, 2 = Danger
int classify(float t, int water, int soil, int rain) {
  int level = 0;
  if (t >= 35)           level = max(level, 2);
  else if (t >= 30)      level = max(level, 1);
  if (water > 1500)      level = max(level, 2);
  else if (water > 700)  level = max(level, 1);
  if (soil > 3500)       level = max(level, 2);
  else if (soil >= 2000) level = max(level, 1);
  if (rain == 0)         level = max(level, 1);  // 0 = rain detected
  return level;
}
```

Listing 3 shows how the level drives the outputs, and Listing 4 shows how the readings are published to the cloud.

Listing 3: Output control (representative implementation)

```
void updateOutputs(int level) {
  digitalWrite(GREEN_LED,  level == 0);
  digitalWrite(YELLOW_LED, level == 1);
  digitalWrite(RED_LED,    level == 2);
  if (level == 2) { /* short tone burst on the buzzer pin */ }
  else            { /* buzzer silent */ }
}
```

Listing 4: Publishing readings to Blynk

```
Blynk.virtualWrite(V0, temperature);   // degrees Celsius
Blynk.virtualWrite(V1, humidity);      // relative humidity, %
Blynk.virtualWrite(V2, waterValue);    // raw ADC count
Blynk.virtualWrite(V3, soilValue);     // raw ADC count
Blynk.virtualWrite(V4, rainState);     // 0 = rain, 1 = no rain
```

4.4 Communication Protocol

Three different links are involved. **Sensors to ESP32:** the DHT22 uses a proprietary single-wire digital protocol handled by the DHT library, the water and soil sensors present analog voltages that the ADC converts to counts, and the rain module presents a plain logic level. **ESP32 to computer:** a UART serial connection over USB at 115200 baud carries the debug output and is also used for programming. The monitor and the firmware must use the same baud rate, otherwise the text appears garbled. **ESP32 to cloud:** the board joins a Wi-Fi network (IEEE 802.11, 2.4 GHz) and the Blynk library keeps a persistent TCP/IP connection to Blynk Cloud, authenticated with the device token, over which `virtualWrite` messages are sent.

4.5 IoT Platform: Blynk

Blynk is the cloud platform used for remote monitoring. A template named "IoT Environmental Monitor" holds five datastreams. A virtual pin is a logical channel between the program and the dashboard and has no connection to a physical GPIO pin [7]; all virtual pin values are transferred as text and interpreted by the widget that displays them. Table 9 lists the datastreams.

Table 9: Blynk virtual pin assignment

Virtual pin	Quantity	Unit / meaning
V0	Temperature	°C
V1	Humidity	Relative humidity, %
V2	Water level	Raw ADC count (0–4095)
V3	Soil moisture	Raw ADC count (high = dry)
V4	Rain	0 = rain detected, 1 = no rain

Gauges suit V0 and V1, value displays or charts suit V2 and V3, and a labelled value or LED widget suits V4. **Credential handling:** the Blynk token and the Wi-Fi password are secrets. They should be kept in a separate header file that is excluded from version control, and the token should be regenerated in the Blynk console if it is ever exposed.

Chapter 5: Results and Discussion

5.1 Experimental Setup

Figure 4 shows the assembled prototype: the ESP32 DevKit V1, breadboards, the soil probe, the rain and water sensor boards, the comparator modules, LEDs and buzzer. The board was powered and programmed through USB from a laptop running VS Code and PlatformIO, the serial monitor was set to 115200 baud, and the node connected to a Wi-Fi network and to the Blynk template described in Section 4.5.

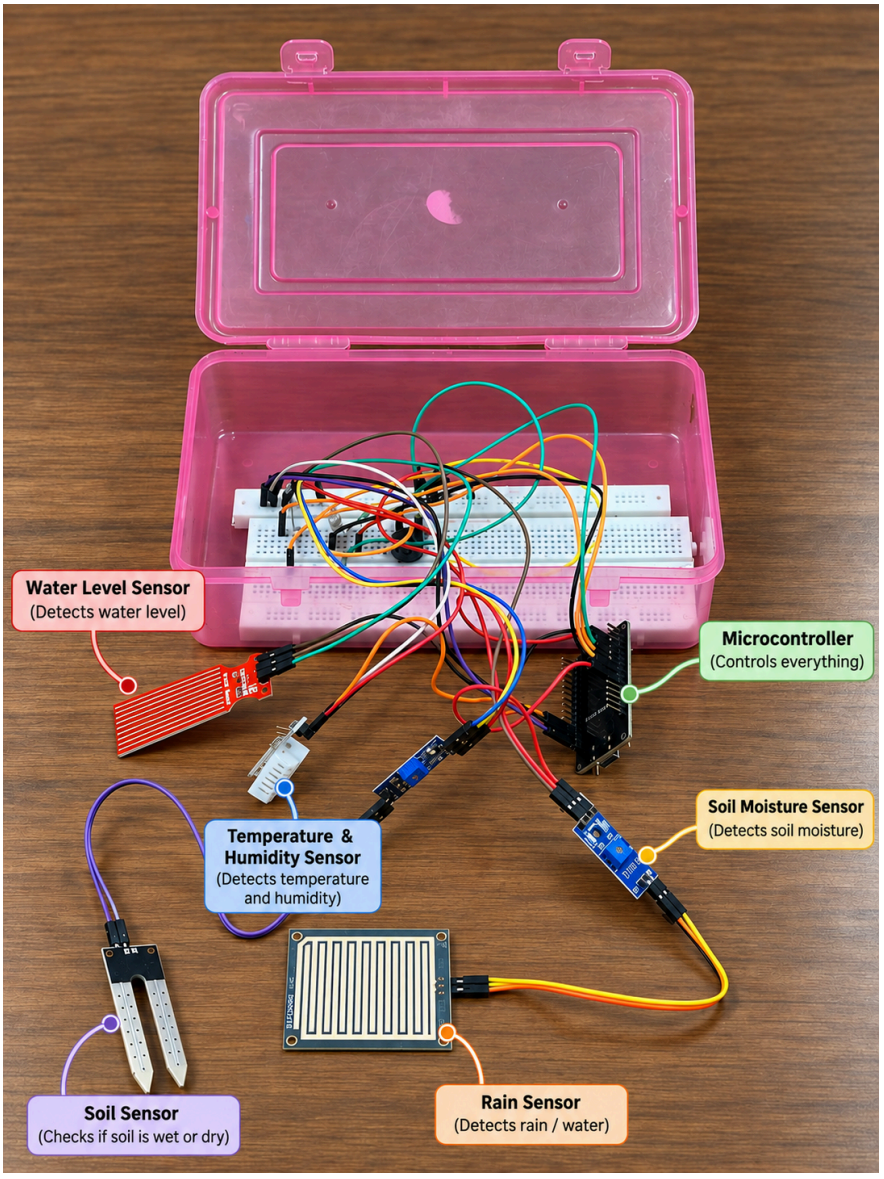


Figure 4: Assembled hardware prototype

5.2 Output Screenshots

Figures 5 and 6 are reserved for the serial monitor output and the Blynk dashboard captured during the demonstration.



Figure 5: Serial monitor output showing live sensor readings

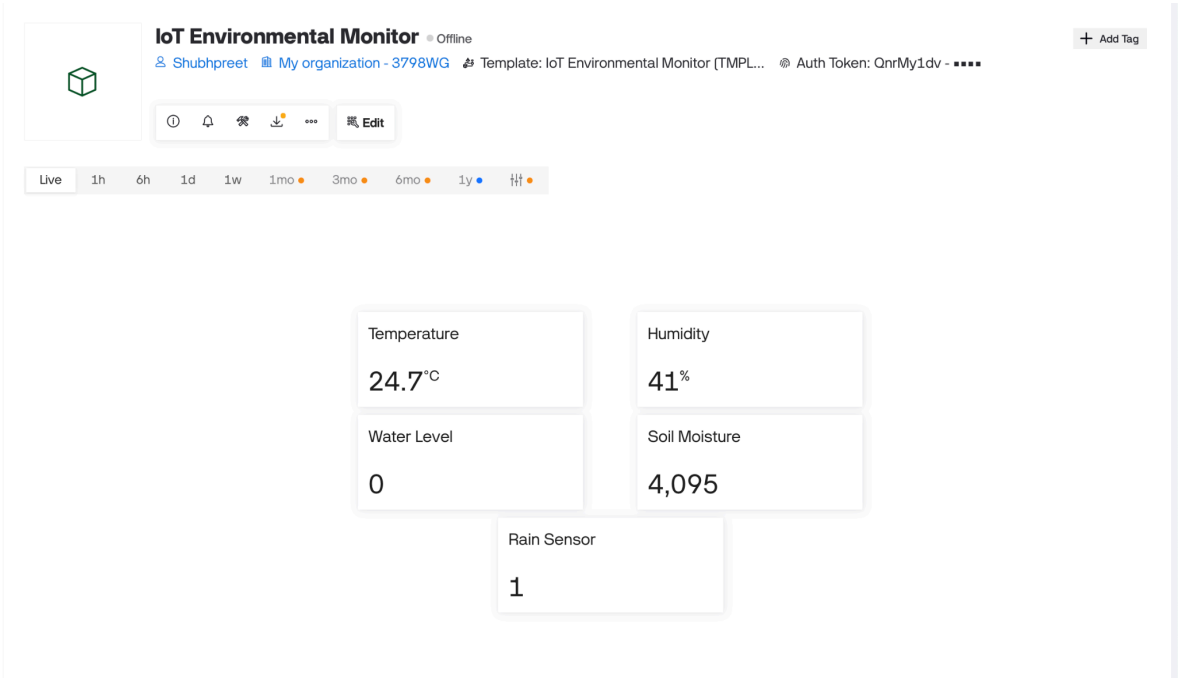


Figure 6: Blynk dashboard with temperature, humidity, water, soil and rain widgets

5.3 Test Cases

Table 10 lists the test cases that verify the logic of Section 3.6. Each case states the condition applied and the response the system is designed to give, and each can be reproduced by exposing the sensors to the corresponding condition (for example wetting the soil probe, wetting the rain plate, or warming the DHT22).

Table 10: Test cases and expected system response

No.	Condition applied	Expected level	Expected outputs
1	T = 27 °C, water 400, soil 1300, no rain	Normal	Green LED on; buzzer off
2	T = 32 °C, others normal	Warning	Yellow LED on; buzzer off
3	T = 36 °C, others normal	Danger	Red LED on; short beep
4	Water reading 900, others normal	Warning	Yellow LED on
5	Water reading 1600, others normal	Danger	Red LED on; short beep
6	Soil reading 2600, others normal	Warning	Yellow LED on
7	Soil reading 4095 (probe in air), others normal	Danger	Red LED on; short beep
8	Rain detected (DO = 0), others normal	Warning	Yellow LED on
9	T = 32 °C, water 1600, rain detected	Danger	Highest severity wins: red LED, beep
10	DHT22 returns an invalid value	—	Error message printed; V0 and V1 not updated; invalid value not used

5.4 Observations

Table 11 records how each sensor behaved in the prototype. The soil values are practical observations from this setup and not laboratory calibration figures.

Table 11: Observed sensor behaviour

Sensor / output	Observation
FC-28 soil probe (AO)	About 4095 in air or very dry condition; roughly 1250 to 1350 when wet
Rain module (DO)	0 when rain is detected; 1 when the plate is dry
DHT22	Valid temperature and humidity every cycle; invalid readings are caught with isnan()
Passive buzzer	Sounds only when driven with a tone (PWM), not with a steady HIGH level
Serial monitor	Readable at 115200 baud once the correct port and speed are selected
Blynk datastreams	V0 to V4 updated by the firmware at the two-second interval

Several practical problems were met and solved during development. They are summarised in Table 12 because they illustrate how the toolchain and the hardware behave in practice.

Table 12: Problems encountered and their resolution

Problem	Cause	Resolution
Serial monitor showed garbled characters	Monitor settings did not match the data sent (port or baud rate)	Select the correct port and use 115200 baud in both firmware and monitor
`Arduino.h` not found when running main.cpp with Code Runner	Code Runner calls plain g++, which has no ESP32 framework files	Build, upload and monitor through PlatformIO
Buzzer silent or weak	A passive buzzer needs an oscillating signal	Generate a tone through PWM instead of a steady HIGH
Soil reading stuck near 4095	Probe in air or very dry, ADC near its upper limit	Treat as very dry; set thresholds from observed wet and dry values
Different serial port names on Windows and macOS	Operating systems name USB serial devices differently (COM3 versus /dev/cu.usbserial-0001)	Choose the matching port for the computer in use

5.5 Discussion

Highest-severity rule. Taking the maximum grade makes the system conservative: a single hazardous parameter is enough to raise the alarm. The cost is that the overall level does not say which parameter caused it, so the individual values on the dashboard remain important.

Sampling interval. Two seconds is frequent enough to follow changes in these slowly varying quantities and matches the response time of the DHT22 [2]. Faster polling would add network traffic without adding information.

ADC readings are not physical units. The ESP32 ADC returns counts, and its response is not perfectly linear across the range. Converting counts to engineering units needs calibration. For a rough illustration, the input voltage is approximately $V = (\text{count} / 4095) \times 3.3$ V, and soil moisture could be expressed as a percentage by mapping between a dry and a wet reference: $\text{moisture \%} = (\text{dry} - \text{count}) / (\text{dry} - \text{wet}) \times 100$. Using the observed extremes dry = 4095 and wet ≈ 1300, a count of 2700 corresponds to about 50 %.

The same mapping shows why calibration matters. The Warning limit of 2000 corresponds to roughly 75 % moisture and the Danger limit of 3500 to about 21 %, so the present limits are strongly tied to the wet reference used during the demonstration. They give a clear, stable demonstration but are not agronomic recommendations.

5.6 Limitations Observed During Testing

- Thresholds are fixed prototype values and have not been verified against known dry and wet soil samples or measured water depths.
- The FC-28 is a resistive probe, and resistive probes corrode with use, so readings may drift over time.
- The readings were taken on a bench prototype; behaviour outdoors, in direct sunlight or in rain has not been characterised.
- Sensor accuracy rests on the manufacturer's specifications [8]; independent checks against a reference instrument are left as future work.

Chapter 6: Conclusion and Future Scope

6.1 Summary of Achievements

- Built a working ESP32 node that measures temperature, humidity, water level, soil moisture and rain.
- Implemented a transparent threshold and highest-severity rule that grades the situation as Normal, Warning or Danger.
- Provided immediate local indication through green, yellow and red LEDs and a PWM-driven buzzer in the Danger state.
- Streamed all five readings to a Blynk dashboard through virtual pins V0 to V4 at two-second intervals.
- Documented the pin mapping, firmware flow, tools, and the problems solved, so the prototype can be reproduced.

6.2 Conclusion

The project demonstrates that a low-cost ESP32 node can combine multi-sensor acquisition, decision making at the edge and cloud visualisation in one compact design. The reviewed literature shows many single-purpose Blynk systems; this work shows how their parameters can be unified under one easily understood warning scheme with both local and remote output. The honest summary is that the system is a functional prototype whose thresholds are practical rather than calibrated, and the report therefore presents calibration as the most important next step.

6.3 Limitations

- **Cloud dependence of the dashboard:** remote viewing needs Wi-Fi and internet access. The local alerts are designed to work independently, but the standard Blynk connection call waits until it connects, so a production version should use non-blocking connection handling.
- **Security:** credentials are stored in the source file in the prototype, which is acceptable for a classroom demonstration but not for deployment.
- **No actuation or logging:** the node does not act on the environment and keeps no history beyond what the cloud stores.

- **Packaging:** the circuit is on breadboards, with no enclosure or protection against moisture, and it depends on USB power.

6.4 Future Enhancements

- Calibrate each sensor with known dry and wet soil and measured water depths, and derive thresholds from the data.
- Replace the resistive soil probe with a capacitive one, which resists corrosion, and consider an ultrasonic or float sensor for water level.
- Add an LCD or OLED display and Blynk push notifications so that alerts reach the user away from the device.
- Use non-blocking Wi-Fi and Blynk connection logic (`Blynk.config` with `Blynk.connect()`) and automatic reconnection.
- Log data to storage or a spreadsheet and apply data-science methods, for example anomaly detection or short-term forecasting of soil and temperature trends.
- Add relay-driven actuators such as a pump or fan for closed-loop control once the thresholds are validated.
- Reduce power use with deep sleep and a battery and solar supply, and place the electronics in a weatherproof enclosure.
- Move credentials to a separate untracked file and support over-the-air firmware updates.

References

[1] D. Babic, I. Jovovic, T. Popovic, N. Kovac, and S. Cakic, “An Internet of Things system for environmental monitoring based on ESP32 and Blynk,” in *Proc. 2022 26th Int. Conf. Inf. Technol. (IT)*, IEEE, Feb. 2022, pp. 1–5.

[2] A. A. Adeagbo, “IoT based environment monitoring system using ESP32,” arXiv:2405.14047, 2024. [Online]. Available: https://arxiv.org/abs/2405.14047

[3] Z. Faysal and G. Mohammed, “Remote farm monitoring and irrigation system,” *Al-Rafidain J. Comput. Sci. Math.*, vol. 15, no. 2, pp. 123–138, 2021, doi: 10.33899/csmj.2021.170016.

[4] N. I. Mohd Najid and R. Omar, “IoT-based plant watering system,” *Evol. Elect. Electron. Eng.*, vol. 5, no. 1, pp. 397–402, 2024. [Online]. Available: https://periodical.uthm.edu.my/index.php/eeee/article/view/15356

[5] J. Lias and G. Kumaran, “IoT-based flood monitoring system using ESP32,” *Evol. Elect. Electron. Eng.*, vol. 5, no. 1, pp. 470–478, 2024. [Online]. Available: https://periodical.uthm.edu.my/index.php/eeee/article/view/15390

[6] Espressif Systems, “ESP32 Wi-Fi & Bluetooth SoC.” [Online]. Available: https://www.espressif.com/en/products/socs/esp32 (accessed Oct. 5, 2026).

[7] Blynk, “Virtual pins,” *Blynk IoT Documentation*. [Online]. Available: https://docs.blynk.io/en/blynk-library-firmware-api/virtual-pins (accessed Oct. 5, 2026).

[8] Aosong Electronics, “Digital-output relative humidity & temperature sensor/module: DHT22 (AM2302),” datasheet.

[9] PlatformIO, “PlatformIO documentation.” [Online]. Available: https://docs.platformio.org

[10] Espressif Systems, “Arduino-ESP32 documentation: ADC and LEDC peripherals.” [Online]. Available: https://docs.espressif.com/projects/arduino-esp32

[11] Blynk, “Labeled value,” *Blynk IoT Documentation*. [Online]. Available: https://docs.blynk.io/en/blynk.apps/widgets-displays/labeled-value (accessed Oct. 5, 2026).

Appendix

Appendix A: Complete Source Code

The complete `src/main.cpp` and `platformio.ini` of the project are to be listed here. Replace the Wi-Fi password and Blynk authentication token with placeholders before including the code.

Main.cpp

```
#define BLYNK_TEMPLATE_ID " "
#define BLYNK_TEMPLATE_NAME "IoT Environmental Monitor"
#define BLYNK_AUTH_TOKEN " "


#include <Arduino.h>
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <DHTesp.h>


// ================= WIFI =================


char ssid[] = " ";
char pass[] = " ";
// ======================================================
// DHT22
// ======================================================


#define DHT_PIN 15


// DHTesp instead of Adafruit DHT: Adafruit's ESP32 path disables
// interrupts for an unbounded time and trips the interrupt watchdog.
DHTesp dht;


// ======================================================
```

```
// SENSORS
// ======================================================

#define WATER_PIN 34
#define SOIL_PIN 35
#define RAIN_DO 27

// ======================================================
// OUTPUTS
// ======================================================

#define RED_LED 25
#define YELLOW_LED 26
#define GREEN_LED 33
#define BUZZER 32

// ======================================================
// BUZZER SETTINGS
// ======================================================

#define BUZZER_CHANNEL 0
#define BUZZER_FREQUENCY 2000
#define BUZZER_RESOLUTION 8

// ======================================================
// BLYNK TIMER
// ======================================================

BlynkTimer timer;

// ======================================================
```

```
// BUZZER FUNCTIONS
// ========================================================


void buzzerOn()
{
   ledcWriteTone(BUZZER_CHANNEL, BUZZER_FREQUENCY);
}


void buzzerOff()
{
   ledcWriteTone(BUZZER_CHANNEL, 0);
}


// ========================================================
// SEND SENSOR DATA
// ========================================================


void sendSensorData()
{
   // ------------------------------------------------
   // READ SENSORS
   // ------------------------------------------------


   TempAndHumidity th = dht.getTempAndHumidity();
   float temperature = th.temperature;
   float humidity = th.humidity;


   int waterValue = analogRead(WATER_PIN);
   int soilValue = analogRead(SOIL_PIN);
   int rainState = digitalRead(RAIN_DO);
```

```
// ------------------------------------------------
// SERIAL OUTPUT
// ------------------------------------------------

Serial.println();
Serial.println("========== SENSOR READINGS ==========");

if (!isnan(temperature))
{
    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" C");

    Blynk.virtualWrite(V0, temperature);
}
else
{
    Serial.println("Temperature: ERROR");
}

if (!isnan(humidity))
{
    Serial.print("Humidity: ");
    Serial.print(humidity);
    Serial.println(" %");

    Blynk.virtualWrite(V1, humidity);
}
else
{
    Serial.println("Humidity: ERROR");
```

```
  }

  Serial.print("Water Level: ");
  Serial.println(waterValue);

  Serial.print("Soil Moisture: ");
  Serial.println(soilValue);

  Serial.print("Rain: ");
  Serial.println(rainState);

  // ------------------------------------------------
  // BLYNK
  // ------------------------------------------------

  Blynk.virtualWrite(V2, waterValue);
  Blynk.virtualWrite(V3, soilValue);
  Blynk.virtualWrite(V4, rainState);

  // ================================================
  // WARNING LEVEL
  //
  // 0 = NORMAL
  // 1 = WARNING
  // 2 = DANGER
  // ================================================

  int warningLevel = 0;

  // ================================================
  // TEMPERATURE
```

```
// ================================================

if (!isnan(temperature))
{
    if (temperature >= 35)
    {
        warningLevel = 2;
    }
    else if (temperature >= 30)
    {
        warningLevel = 1;
    }
}

// ================================================
// WATER LEVEL
// ================================================
//
// Based on your observed readings:
//
// 0-700       = Normal
// 701-1500    = Warning
// >1500       = Danger
//
// ================================================

if (waterValue > 1500)
{
    warningLevel = 2;
}
else if (waterValue > 700 && warningLevel < 1)
```

```
{
    warningLevel = 1;
}

// ================================================
// SOIL MOISTURE
// ================================================
//
// Your FC-28:
//
// ~4095 = very dry
// ~1250-1350 = wet
//
// 0-1999     = Normal
// 2000-3500  = Warning
// >3500      = Danger
//
// ================================================

if (soilValue > 3500)
{
    warningLevel = 2;
}
else if (soilValue >= 2000 && warningLevel < 1)
{
    warningLevel = 1;
}

// ================================================
// RAIN
// ================================================
```

```
//
// 0 = Rain detected
// 1 = No rain
//
// Rain produces WARNING, not danger.
//
// ================================================


if (rainState == 0)
{
    Serial.println("Rain Status: RAIN DETECTED");


    if (warningLevel < 1)
    {
        warningLevel = 1;
    }
}
else
{
    Serial.println("Rain Status: NO RAIN");
}


// ================================================
// RESET OUTPUTS
// ================================================


digitalWrite(RED_LED, LOW);
digitalWrite(YELLOW_LED, LOW);
digitalWrite(GREEN_LED, LOW);


buzzerOff();
```

```
// ================================================
// NORMAL
// ================================================

if (warningLevel == 0)
{
    digitalWrite(GREEN_LED, HIGH);

    Serial.println("SYSTEM STATUS: NORMAL");
}

// ================================================
// WARNING
// ================================================

else if (warningLevel == 1)
{
    digitalWrite(YELLOW_LED, HIGH);

    Serial.println("SYSTEM STATUS: WARNING");
}

// ================================================
// DANGER
// ================================================

else
{
    digitalWrite(RED_LED, HIGH);
```

```
        Serial.println("SYSTEM STATUS: DANGER");


        // Short buzzer beep
        buzzerOn();
        delay(200);
        buzzerOff();
    }

    Serial.println("====================================");
}

// ======================================================
// CONNECTION CHECK
// ======================================================

// Print why the ESP32 dropped / failed to join, so the serial log
// tells us whether it's a password, signal or AP problem.
void onWiFiDisconnected(WiFiEvent_t event, WiFiEventInfo_t info)
{
    uint8_t reason = info.wifi_sta_disconnected.reason;
    Serial.print("\nWiFi: disconnected, reason ");
    Serial.print(reason);

    switch (reason)
    {
    case 2:
    case 15:
    case 202:
    case 204:
            Serial.println(" (auth failed - check password / network
type)");
        break;
```

```
   case 201:
           Serial.println(" (SSID not found - out of range or 5 GHz
only)");
       break;
   case 203:
           Serial.println(" (AP refused association - AP full or MAC
blocked)");
       break;
   default:
       Serial.println();
       break;
   }
}

// List every AP broadcasting our SSID with its signal strength.
void scanForNetwork()
{
   Serial.println("WiFi: scanning...");
   int n = WiFi.scanNetworks();
   bool found = false;

   for (int i = 0; i < n; i++)
   {
       if (WiFi.SSID(i) == ssid)
       {
           found = true;
           Serial.printf("  found \"%s\"  RSSI %d dBm  ch %d  %s\n",
                         ssid, WiFi.RSSI(i), WiFi.channel(i),
                                             WiFi.encryptionType(i)  ==
WIFI_AUTH_WPA2_ENTERPRISE
                                    ? "ENTERPRISE (not supported by this
code)"
                                : "");
```

```
        }
    }

    if (!found)
    {
         Serial.printf("  \"%s\" NOT visible (%d networks seen)\n", ssid,
n);
    }

    WiFi.scanDelete();
}

void checkConnection()
{
    static unsigned long lastReconnect = 0;

    if (WiFi.status() != WL_CONNECTED)
    {
            //  A  WPA2  handshake  on  a  busy  network  can  take  >5  s;
reconnecting
        // too often aborts every attempt before it can finish.
        if (millis() - lastReconnect >= 20000)
        {
            Serial.println("WiFi: NOT CONNECTED (retrying...)");
            lastReconnect = millis();
            WiFi.disconnect();
            WiFi.begin(ssid, pass);
        }
        return;
    }

    if (!Blynk.connected())
```

```
    {
        Serial.print("WiFi: CONNECTED (IP ");
        Serial.print(WiFi.localIP());
        Serial.println(") - connecting to Blynk...");

        if (Blynk.connect(3000))
        {
            Serial.println("Blynk: CONNECTED");
        }
        else
        {
              Serial.println("Blynk: FAILED (check auth token / internet
access)");
        }
    }
}

// ========================================================
// SETUP
// ========================================================

void setup()
{
    Serial.begin(115200);
    delay(1000);

    // DHT
    dht.setup(DHT_PIN, DHTesp::DHT22);

    // Sensors
    pinMode(WATER_PIN, INPUT);
    pinMode(SOIL_PIN, INPUT);
```

```
  pinMode(RAIN_DO, INPUT);

  // LEDs
  pinMode(RED_LED, OUTPUT);
  pinMode(YELLOW_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);

  // Buzzer
  pinMode(BUZZER, OUTPUT);

  // Buzzer PWM
  ledcSetup(BUZZER_CHANNEL, BUZZER_FREQUENCY, BUZZER_RESOLUTION);
  ledcAttachPin(BUZZER, BUZZER_CHANNEL);
  buzzerOff();

  // LEDs OFF
  digitalWrite(RED_LED, LOW);
  digitalWrite(YELLOW_LED, LOW);
  digitalWrite(GREEN_LED, LOW);

  // ================================================
  // BLYNK
  // ================================================

  Serial.println("================================");
  Serial.println("IoT ENVIRONMENTAL MONITOR");
  Serial.println("Connecting to Blynk...");
  Serial.println("================================");

  // Non-blocking: Blynk.begin() would hang here forever if WiFi
  // or Blynk is unreachable, and the sensors/LEDs would never run.
```

```
  WiFi.mode(WIFI_STA);
  WiFi.setSleep(false);
                                        WiFi.onEvent(onWiFiDisconnected,
ARDUINO_EVENT_WIFI_STA_DISCONNECTED);

  // Campus networks have many APs with the same SSID. Scan all
  // channels and join the strongest one instead of the first found.
  WiFi.setScanMethod(WIFI_ALL_CHANNEL_SCAN);
  WiFi.setSortMethod(WIFI_CONNECT_AP_BY_SIGNAL);

  scanForNetwork();

  WiFi.begin(ssid, pass);
  Blynk.config(BLYNK_AUTH_TOKEN);

  // Connect immediately instead of waiting for the first timer tick.
  // Wait at most 20 s for WiFi so sensors still start if it's down.
  Serial.print("WiFi: connecting");
  unsigned long wifiStart = millis();
    while (WiFi.status() != WL_CONNECTED && millis() - wifiStart <
20000)
  {
      delay(250);
      Serial.print(".");
  }
  Serial.println();
  checkConnection();

  // Send data every 2 seconds
  timer.setInterval(2000L, sendSensorData);

  // Check / restore connection every 5 seconds
```

```
   timer.setInterval(5000L, checkConnection);
}

// ====================================================
// LOOP
// ====================================================

void loop()
{
   if (Blynk.connected())
   {
       Blynk.run();
   }
   timer.run();
}
```

Platformio.ini

```
; PlatformIO Project Configuration File
;
;   Build options: build flags, source filter
;   Upload options: custom upload port, speed and extra flags
;   Library options: dependencies, extra library storages
;   Advanced options: extra scripting
;
; Please visit documentation for the other options and examples
; https://docs.platformio.org/page/projectconf.html

[env:IoT_Environmental_Monitor]
platform = espressif32
```

```
board = esp32dev

framework = arduino

monitor_speed = 115200

lib_deps =

 beegee-tokyo/DHT sensor library for ESPx

 blynkkk/Blynk
```

Appendix B: Quick Reference

Table A1: Summary of configuration values

Item	Value
Controller	ESP32 DevKit V1
DHT22 / Water / Soil AO / Rain DO	GPIO15 / GPIO34 / GPIO35 / GPIO27
Red / Yellow / Green LED	GPIO25 / GPIO26 / GPIO33 (each with 220 Ω)
Buzzer	GPIO32 (tone / PWM)
Serial speed	115200 baud
Update interval	2 seconds
Blynk virtual pins	V0 temperature, V1 humidity, V2 water, V3 soil, V4 rain
Temperature limits	Warning 30 °C, Danger 35 °C
Water limits (ADC)	Warning above 700, Danger above 1500
Soil limits (ADC)	Warning from 2000, Danger above 3500
Rain logic	0 = rain detected (Warning), 1 = no rain

Plagiarism Report

IoT_Smart_Irrigation&Warning_System_Final_REPORT-2.pdf

Shri Vile Parle Kelavani Mandal

Document Details

Submission ID
trn:oid:::9832:153790006

Submission Date
Oct 6, 2026, 10:29 AM GMT+5:30

Download Date
Oct 6, 2026, 10:31 AM GMT+5:30

File Name
IoT_Smart_Irrigation&Warning_System_Final_REPORT-2.pdf

File Size
4.7 MB

47 Pages

7,408 Words

48,541 Characters

16% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

79 Not Cited or Quoted 13%
Matches with neither in-text citation nor quotation marks

5 Missing Quotations 1%
Matches that are still very similar to source material

19 Missing Citation 3%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

10% Internet sources
5% Publications
15% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **79** Not Cited or Quoted 13%
  Matches with neither in-text citation nor quotation marks
- **5** Missing Quotations 1%
  Matches that are still very similar to source material
- **19** Missing Citation 3%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

10% Internet sources
5% Publications
15% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	wokwi.com 1%
2	Internet	community.platformio.org <1%
3	Submitted works	University of Glasgow on 2026-05-16 <1%
4	Internet	community.blynk.cc <1%
5	Submitted works	Baze University on 2026-03-24 <1%
6	Submitted works	BB9.1 PROD on 2026-05-01 <1%
7	Publication	Muhamad Zefa Heryana, Zery Gallanta Sasongko, Mochammad Haldi Widianto. "... <1%
8	Internet	dspace.daffodilvarsity.edu.bd:8080 <1%
9	Submitted works	October University for Modern Sciences and Arts (MSA) on 2026-06-16 <1%
10	Internet	titula.universidadeuropea.es <1%

11 Submitted works
Engineers Australia on 2020-10-05 <1%

12 Internet
www.hackster.io <1%

13 Submitted works
University of Hertfordshire on 2026-03-24 <1%

14 Submitted works
University of Stellenbosch, South Africa on 2026-06-01 <1%

15 Internet
forum.arduino.cc <1%

16 Submitted works
Bilkent University on 2025-11-02 <1%

17 Submitted works
FEGT on 2026-09-01 <1%

18 Submitted works
Universidad Internacional de la Rioja on 2026-04-17 <1%

19 Submitted works
Politeknik Merlimau (PMM) on 2025-11-14 <1%

20 Submitted works
Universiti Malaysia Perlis on 2026-06-25 <1%

21 Submitted works
Institute of International Studies on 2023-04-10 <1%

22 Submitted works
Munster Technological University (MTU) on 2025-05-23 <1%

23 Submitted works
University of Cyberjaya on 2026-07-06 <1%

24 Submitted works
East Delta University on 2026-09-08 <1%

25 Internet

meisadi.blogspot.com <1%

26 Submitted works

Coventry University on 2026-08-18 <1%

27 Submitted works

De Montfort University on 2022-10-02 <1%

28 Submitted works

University of Northumbria at Newcastle on 2026-05-18 <1%

29 Internet

tutorial.cytron.io <1%

30 Internet

ubir.buffalo.edu <1%

31 Submitted works

Islington College,Nepal on 2026-01-16 <1%

32 Submitted works

University of Central Lancashire on 2026-03-31 <1%

33 Submitted works

The University of the South Pacific on 2026-06-15 <1%

34 Submitted works

University of Cyberjaya on 2026-07-02 <1%

35 Submitted works

University of Sunderland on 2025-05-02 <1%

36 Internet

www.ece.ucf.edu <1%

37 Submitted works

Asia Pacific University College of Technology and Innovation (UCTI) on 2026-09-04 <1%

38 Submitted works

Oklahoma State University on 2012-04-18 <1%

39 Submitted works
University of South Africa on 2025-09-30 <1%

40 Internet
aclweb.org <1%

41 Submitted works
Arab Open University on 2024-11-07 <1%

42 Submitted works
Durban University of Technology on 2024-06-18 <1%

43 Submitted works
Jacobs University, Bremen on 2026-06-30 <1%

44 Submitted works
University of Derby on 2026-05-10 <1%

45 Internet
odr.chalmers.se <1%

46 Internet
sciencedocbox.com <1%

47 Internet
tesi.cab.unipd.it <1%

48 Internet
www.coursehero.com <1%

49 Internet
www.electroniclinic.com <1%

50 Submitted works
General Sir John Kotelawala Defence University on 2026-06-11 <1%

51 Submitted works
University Tun Hussein Onn Malaysia on 2025-01-23 <1%

52 Internet
bspace.buid.ac.ae <1%

53 Internet
dr.ur.ac.rw <1%

54 Internet
engineering-shirpur.nmims.edu <1%

55 Internet
eprints2.undip.ac.id <1%

56 Internet
krex.k-state.edu <1%

57 Internet
www.nmimsindore.org <1%

58 Submitted works
American University of the Middle East on 2025-05-16 <1%

59 Submitted works
Amity University on 2017-05-22 <1%

60 Submitted works
Birkbeck College on 2024-09-30 <1%

61 Submitted works
Islington College,Nepal on 2026-01-02 <1%

62 Publication
James A. Langbridge. "Arduino™ Sketches", Wiley, 2014 <1%

63 Submitted works
Middle East College of Information Technology on 2018-02-03 <1%

64 Submitted works
STEKOM on 2025-08-09 <1%

65 Submitted works
Technological University Dublin on 2026-05-27 <1%

66 Submitted works
Teesside University (Blackboard Ultra) on 2026-06-05 <1%

67 Submitted works
Universiti Kuala Lumpur on 2026-01-26 <1%

68 Submitted works
Universiti Kuala Lumpur on 2026-06-08 <1%

69 Internet
electronic-crafts.blogspot.com <1%

70 Internet
journal.uvers.ac.id <1%

71 Internet
mcgill.scholaris.ca <1%

72 Internet
rickrejeleene.me <1%

73 Internet
www.mdpi.com <1%

74 Publication
Dejan Babic, Ivan Jovovic, Tomo Popovic, Natasa Kovac, Stevan Cakic. "An Interne... <1%

75 Publication
Gaurav Aggarwal, Rishabh Dev Shukla, Innocent Ewean Davidson, Ramesh Bansal... <1%

76 Submitted works
Politeknik Sultan Salahuddin Abdul Aziz Shah (PSA) on 2025-11-14 <1%

77 Submitted works
University of Huddersfield on 2026-05-04 <1%

78 Submitted works
Bournemouth University on 2022-08-22 <1%

79 Submitted works
Durban University of Technology on 2026-06-13 <1%

80 Submitted works
Islington College,Nepal on 2026-01-16 <1%

81 Submitted works

Singapore Institute of Technology on 2026-04-05 <1%

82 Submitted works

Universiti Malaysia Pahang Al-Sultan Abdullah (UMPSA) on 2026-06-14 <1%

83 Submitted works

Vaal University of Technology on 2026-07-10 <1%